Project Requirement Documents

2021113424 김다영

0.Functional Requirements

1.A user can sign up, sign in and sign out via web browser.

2.Any signed or non-signed user can see current coin price and the price trend.

3.When a user creates his/her account, the money amount is 0 won and the number of coins is 0.

4.The marketplace initially has 100 coins, and the coin price is 100 won.

5. The marketplace site should post current coin price and available coins which can be sold from the marketplace.

6. if there are several transactions for coin trading and coin price changes, then the site have to show the price change history through a graph or a table.

7. etc..

1. User Stories

1. As a user, I want to go sign up for the site, so that I can use the marketplace.
2. As a user, I want to deposit money, so that I can buy coins.
3. As a user, I want to post coins for sales, so that others can buy them.
4. As a user, I want to withdraw my remaining money, so that I can get real money.

   User Flow: sign up -> Deposit money -> Buy coin -> Post sell -> Coin gets sold -> Withdraw money

5. As a marketplace, I want to initialize coins and money to 100, so that all coin amounts maintain 100.
6. As a marketplace, I want to allow users to sign up, so that they can use the service.
7. As a marketplace, I want to show current amounts of coins and money, so that users can buy coins from the marketplace no more than amounts of the marketplace's coins.
8. As a marketplace, I want to allow users to deposit money, so that they can purchase coins.
9. As a marketplace, I want to post user's coin selling posts, so that others can see selling posts.
10. As a marketplace, I want to show history about coin price change, so that users can understand several transactions.
11. As a marketplace, I want to show coin prices and trends, so that others can easily check the stance of the coin without signing in.

12. As a marketplace, I want to exchange coins for users'money,so that they can own coins for trading.
13. As a marketplace, I want to let users withdraw their money from the system.

MarketPlace Flow: Sign up -> initialize market -> Deposit money -> Buy coin -> Post coin for sale -> Coin gets sold->Update coin price & real-time chart -> Withdraw money->Check price/trend without login->Show transaction history

2. Use cases

1.

Name of use case	Sign up for a site.
Description	User sign up for a site.
Actor	User, Site
Precondition	1.SIte is running. 2.User didn't sign up for a site before.
Post condition	1.Site shows sign up success message. 2.User Information is uploaded in Site's DB.
Flow	1. User enters the site. 2. User clicks the sign up button. 3. User writes down User Informations and submits them. 4.Site show result of sign up.
Exception	1. In step 3, User Information is duplicated. a. API error code: 409 Conflict. b. Show message. 2. In step 3, ID is duplicated. a. API error code:409 Conflict. b. Show message. c. Rewrite the ID. d. Submit for sign up.
Additional Requirements	1.Validation,terms of use,error message

2.

Name of use case	Deposit money to marketplace.
Description	User deposit money to marketplace.
Actor	User,Marketplace
Precondition	1.Site is running. 2.User didn't sign up for a site before.
Post condition	1.User money balance is increased. 2.User's coin remains unchanged until a coin purchase is made. 3.Deposit record is logged.
Flow	1. User clicks deposit by marketplace deposit button. 2. User writes amount of money. 3. User submits the amount. 4. User's money increases by submitted amount. 5. System confirms deposit and updates balance.
Exception	1. In step step 3, Amount cannot be negative or non-numeric. a. API error code:400 Bad Request. b. Show message. c. Rewrite amount of money. d. submit again.
Additional Requirements	1.Validation,error message.

3.

Name of use case	Buy coin from marketplace.
Description	User buy coin from marketplace.
Actor	User,Marketplace
Precondition	1.Site is running. 2.User didn't sign up for a site before. 3.User deposit money to marketplace so that User has some money.
Post condition	1.User's coin balance increases. 2.User's money balance decreases. 3.Marketplace's coin balance decreases. 4.Marketplace's money balance increases. 5.Transaction is recorded in system log.
Flow	1.User clicks buy coin by marketplace button. 2.User writes amount of coins. 3.User submits the amount.

	4.User's money decreases, and Marketplace's money increases by coin's price. 5.User's coin increases, and Marketplace's coin decreases.
Exception	1. In step 3, if the number of coins requested is greater than the available coins in the marketplace. a.API error code : 400 Bad request. b.Show message. c.Rewrite the amount of coins. d.Submit the amounts.
Additional Requirements	1.Validation,error message. 2. Marketplace shows current coin amounts before step 2.

4.

Name of use case	Post coin selling.
Description	User posts selling coins on Marketplace.
Actor	User,Marketplace.
Precondition	1.User is signed in. 2.User has coins and want to sell..
Post condition	1.Marketplace shows User's coin selling posts.
Flow	1.User clicks the 'Post Sell' button. 2.User writes selling coin amounts. 3.Marketplace shows User's coin selling posts.
Exception	1. In the step 2. if the number of coins posted is greater than in possession of User. a. API error : 400 Bad request. b. Show message. c. Rewrite the amount of coins. d. Submit the amounts.
Additional Requirements	1.Validation,error message. 2.Marketplace shows trade history and current.

5.

Name of use case	Add money
Description	User receives money after the coin sale.
Actor	User,Marketplace
Precondition	1.User is signed in. 2.User posted coin selling. 3.Successful transaction.
Post condition	1.User's money balance is increased.
Flow	1.User's transaction for selling coins. 2.User's money balance is increased. 3.Marketplace's money decreased. 4.Marketplace updates the coin price in real-time.
Exception	1. In step 4,Update coin price should be real time.
Additional Requirements	1.Validation.

(결과를 먼저 반영해주고 **(add money)** 그 후 거래 **history/**시세 변화 보여주기**)**
-> 사용자 관점으로는 돈들어왔는지가 우선
-> 시스템 관점으론 트랜잭션 완료 후 상태변화**,**기록남기

6.

Name of use case	Withdraw money
Description	User withdraws money from the amount of money on site.
Actor	User,Marketplace
Precondition	1.User is signed in. 2.User has some money to withdraw.
Post condition	1.User's money balance is decreased. 2.System logs the withdrawal transaction.
Flow	1.User clicks the 'withdraw money' button. 2.User writes money amounts. 3.User receive the withdrawn amount.
Exception	1. In step 2,if the withdrawal amount is greater than the user's current balance a. API error: 400 Bad request. b. Show error message.

	c.Rewrite the amount of money. d.Submit the amounts.
Additional Requirements	1.Validation,error message. 2.Marketplace shows the money left.

7.

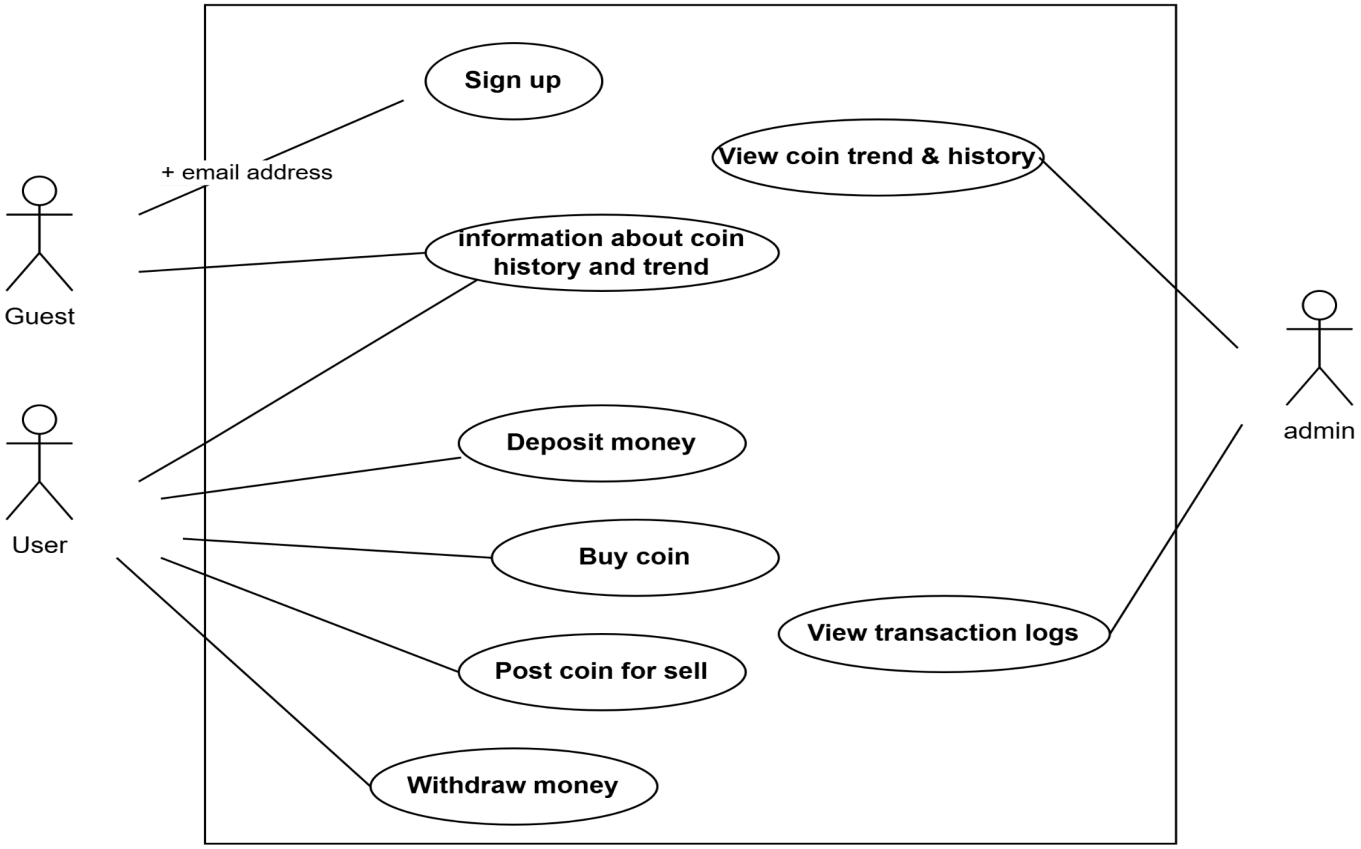
Name of use case	Show coin price and trend.
Description	Marketplace shows coin price and trend.
Actor	Marketplace
Precondition	1.Server is running. 2.Marketplace transparently shows coin price and trend. 3.To show real-time coin prices,Marketplace uses WedSocket. 4. Marketplace needs area to show coin prices and trends.
Post condition	1.Coin price and trend are uploaded and shown on a web site. 2.Coin price and trend are stored in a database. 3.User receives information about coin price and trend also not sign up people too.
Flow	1.User or Marketplace buy or sell the coin. 2.Data is stored in a database. 3. Data mining produces statistics based on database. 4.The marketplace takes information from data mining. 5.Marketplace posts coin and trend and price in web site.
Note	1. In step 2, Use MySQL. 2. In step 4, Use Python(pandas) a. Can be visualized.
Additional Requirements	1.Validation.

8.

Name of use case	Show current amounts of coin and money.
Description	Marketplace shows current amounts of coins and money.
Actor	Marketplace
Precondition	1.Server is running. 2.Marketplace has a section to display current amounts of coin and money.
Post condition	1.The current amounts of coins and money are uploaded and shown in the web browser.

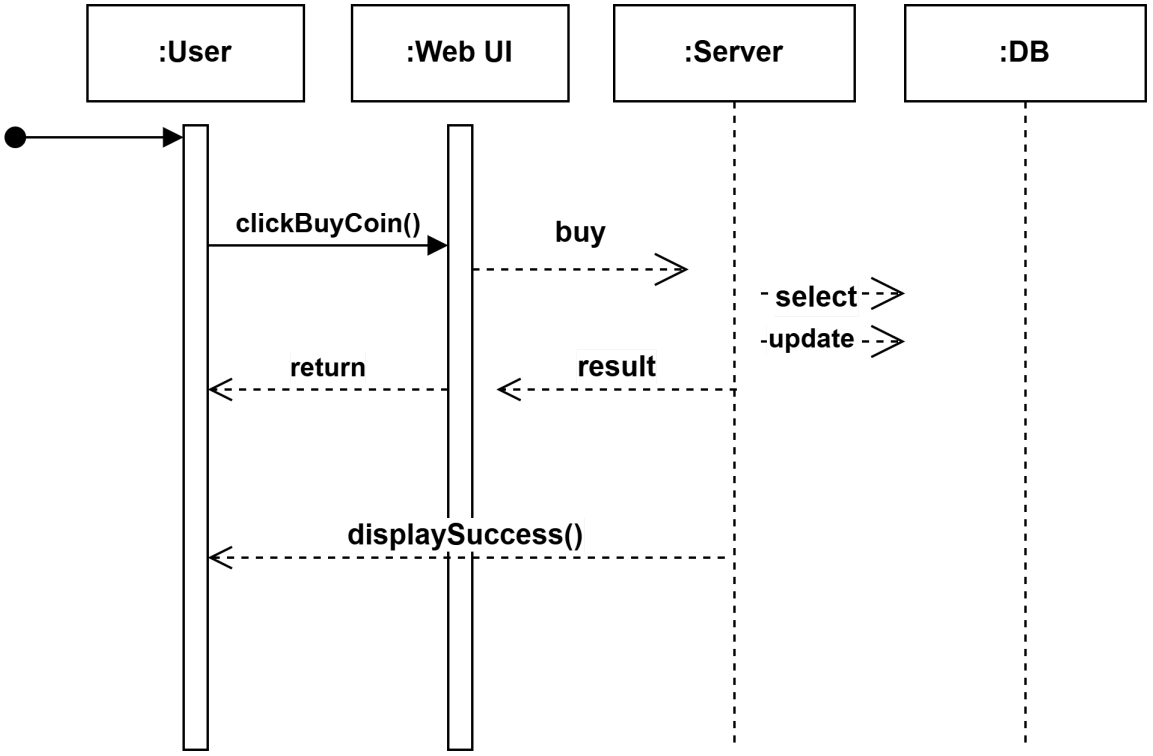
	2.The current amounts of coins and money are stored in a database. 3.User receives information about current amounts of coin and money.
Flow	1.The user trades on the website. 2.Trade data is stored in a database(MySQL). 3.Marketplace posts the amount of coins and money changed based on the database.
Exception	1. In step 3, failure in displaying 'the amount of coin and money based on the database' web page. a. Show error message with returning back to my page section.
Additional Requirements	1. Current amounts of coin and money need to be shown in real - time.

3. UML



draw.io 사용

4. Sequence diagram



5. Rest API 명세서

Index	Method	URL	Description
1.	POST	/auth/signup	회원가입
2.	POST	/auth/login	로그인 및 토큰발급
3.	GET	/auth/me	내 정보(프로필,잔고 등)
4.	POST	/trade/deposit	입금
5.	POST	/trade/withdraw	출금
6.	POST	/trade/buy	코인 구매
7.	POST	/trade/sell	코인 판매글 등록
8.	GET	/trade/sell-posts	등록된 판매글 목록
9.	GET	/trade/price	시세 및 트렌드
10.	GET	/trade/chart	차트용 시세 데이터
11.	GET	/admin/users	유저 목록
12.	GET	/admin/logs	거래 로그

6.Rest API (Flask + VS Code)

a. app.py

routes > routes > app.py > ...

```python
from flask import Flask
from routes.auth import auth_bp
from routes.trade import trade_bp
from routes.admin import admin_bp

app = Flask(__name__)
app.config['JSON_AS_ASCII'] = False


app.register_blueprint(auth_bp, url_prefix='/auth')
app.register_blueprint(trade_bp, url_prefix='/trade')
app.register_blueprint(admin_bp, url_prefix='/admin')

if __name__ == '__main__':
    app.run(debug=True)

```

b.routes/auth.py

```python

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/signup', methods=['POST'])
def signup():
    data = request.json
    return jsonify({ "status": "success", "message": "User created" })

@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.json
    return jsonify({ "status": "success", "token": "jwt-token-example" })

@auth_bp.route('/me', methods=['GET'])
def get_profile():
    return jsonify({ "email": "user@example.com", "balance": 10000, "coins": 5 })

```

c.routes/trade.py

```python
from flask import Blueprint, request, jsonify

trade_bp = Blueprint('trade', __name__)

@trade_bp.route('/deposit', methods=['POST'])
def deposit():
    return jsonify({ "status": "success", "new_balance": 15000 })

@trade_bp.route('/withdraw', methods=['POST'])
def withdraw():
    return jsonify({ "status": "success", "remaining_balance": 5000 })

@trade_bp.route('/buy', methods=['POST'])
def buy():
    return jsonify({ "status": "success", "new_balance": 9000, "new_coin": 5 })

@trade_bp.route('/sell', methods=['POST'])
def sell():
    return jsonify({ "status": "success", "message": "Sell post created" })

@trade_bp.route('/sell-posts', methods=['GET'])
def sell_posts():
    return jsonify({ "posts": [{"id":1,"coin":3,"price":120}] })

@trade_bp.route('/price', methods=['GET'])
def price():
    return jsonify({ "price": 100, "trend": [100, 102, 105, 101] })

@trade_bp.route('/chart', methods=['GET'])
def chart():
    return jsonify({ "data": [100, 101, 102, 99, 98] })
```

->MySQL 연동을 통해 값은 동적 처리

d. routes/admin.py

```python
from flask import Blueprint, jsonify

admin_bp = Blueprint('admin', __name__)

@admin_bp.route('/users', methods=['GET'])
def get_users():
    return jsonify({ "users": [{"id":1,"email":"admin@test.com"}] })

@admin_bp.route('/logs', methods=['GET'])
def get_logs():
    return jsonify({ "logs": [{"user_id":1, "action":"buy", "amount":5}] })

@admin_bp.route('/ban-user', methods=['POST'])
def ban_user():
    return jsonify({ "status": "success", "message": "User banned" })
```

7.Test Case

1.

Test Case ID	001	**Test Case Description**	Sign up for a site
Created By	Cho yeon jun	**Reviewed By**	Kim da young

Step #	**Prerequisites:**
1	ID is correct but Password is wrong,Password error message is showing.
2	Password is correct but ID is wrong,ID error message is showing
3	ID and Password are correct
4	Login is a success.
5	
...	

Step #	**Test Data**
1	ID ='yeon jun', Password='1111'
2	ID='da young' ,Password='1234'
3	ID= 'yeon jun', Password='1234'
4	
5	
...	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter invalid ID or password.	Warn for invalidpassword and ask reentering.		Not executed
2	Enter valid ID and password.	User's login in website.		Not executed
3				
4				
5				
...				

2.

Test Case ID	002	**Test Case Description**	Buy coins from marketplace
Created By	Cho yeon jun	**Reviewed By**	Kim da young

Step #	Prerequisites:
1	Marketplace doesn't have amounts of coins user wants to buy.
2	Marketplace has no any coin.
3	Marketplace has enough coins user wants to buy.

Step #	Test Data
1	MARKET_COIN = 50 BUY_COIN = 60
2	MARKET_COIN = 0
3	MARKET_COIN = 50 BUY_COIN = 30

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter more coins than marketplace has.	Warn for amount of entered coins.		Not executed
2	Enter whatever but marketplace can not sell any coins.	Warn for market cannot sell any coins.		Not executed
3	Enter enough coins less than market has.	Trade of coins and money.		pass

3.

Test Case ID	003	**Test Case Description**	Post coin selling
Created By	Cho yeon jun	**Reviewed By**	Kim da young

Step #	Prerequisites:
1	Amounts of coins user wants to sell is more than amounts user has.
2	User has no any coin.
3	User has enough coins user want to sell.

Step #	Test Data
1	SELL_COIN = 30 USER_COIN = 10
2	USER_COIN = 0
3	SELL_COIN = 50 USER_COIN = 100

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter more coins user has.	Warn for amount of entered coins.		Not executed
2	Enter whatever but user cannot sell any coins.	Warn for user's empty coin.		Not executed
3	Enter enough coins less than user has.	Success message of posting.		pass

4.

Test Case ID	004	**Test Case Description**	Add money
Created By	Cho yeon jun	**Reviewed By**	Kim da young

Step #	Prerequisites:
1	Current user's money is showing.
2	Amounts of money user wants to add message is showing.
3	Amount of the money after adding is showing.

Step #	Test Data
1	
2	ADD_MONEY = 1000
3	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter amounts of money for adding.	Amount of money is added.		Pass
2				
3				

5.

Test Case ID	005	**Test Case Description**	Withdraw money
Created By	Cho yeon jun	**Reviewed By**	Kim da young

Step #	Prerequisites:
1	Transaction password entering message is showing
2	Transcation password entering message with password error message is showing
3	Action selection message is showing
4	
5	
...	

Step #	Test Data
1	Password= '0000'
2	Password= '1111'
3	Selection='withdraw'
4	
5	
...	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter invalid transaction password.	Warn for invalidpassword and ask reentering.		Not executed
2	Enter valid transaction password.	Show message for selecting action.		Not executed
3	Select 'withdraw'	Show the 'enter the amount to withdraw'		Not executed
4				
5				
...				

6.

Test Case ID	006	**Test Case Description**	Show history of coin price change
Created By	Kim da young	**Reviewed By**	Cho yeon jun

Step #	Prerequisites:
1	Marketplace and users buy or sell the coin.
2	Find history of coin price change.
3	Show history of coin price change
4	
5	
...	

Step #	Test Data Requirement
1	Coin price change
2	Visualization data:graph,chart,table.
3	Shows visualized data according to history of coin price change.
4	
5	
...	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Coin price changs	Data is stored in a database.		Not executed
2	Data mining is running	Data mining is analyzed and statistics based on the database.		Not executed
3	Verify that the page displays history of coin price change	The trend graphs illustrate history of coin price change	The graphs show the historical coin price changes.	Not executed
4				
5				
...				

Verify that the marketplace can show users the history of coin price change.

7.

Test Case ID	007	**Test Case Description**	Show coin price and trend
Created By	Kim da young	**Reviewed By**	Cho yeon jun

Step #	Prerequisites:
1	Marketplace and users buy or sell the coin.
2	Implementation of functionality to display coin prices and trends based on different time periods.
3	Show coin price and trend
4	
5	
...	

Step #	Test Data Requirement
1	Coin price change
2	-Coin price data :include various coin prices from different time periods. -Trend data :upward,downward and stable
3	Visualization data:graph,chart,table.
4	Shows visualized data according to coin price and trend
5	
...	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Coin price changs	Data is stored in a database.		Not executed
2	Data mining is running	Data mining is analyzed and statistics based on the database.		Not executed
3	Verify that the page displays the current coin prices and trends for various coins.	The trend graphs illustrate the price changes.		Not executed
4	Coin prices and trends not published	Show the message for try again		Not executed
5				
...				

Marketplace checks to see if it can show users the price and trend of coins.

8.

Test Case ID	008	**Test Case Description**	Post user's coin selling post
Created By	Kim da young	**Reviewed By**	Cho yeon jun

Step #	Prerequisites:
1	Sever is running.
2	The marketplace platform needs a database that can store and manage coin-selling posts.
3	The marketplace must be able to publish user coin selling post.
4	Post user's coin selling post
5	
...	

Step #	Test Data Requirement
1	User information:sales price,quantity,seller information
2	Visualization data:graph,chart,table.
3	Shows visualized data according to the user's coin selling post.
4	
5	
...	

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Post user's coin selling post	Posted immediately	Posted immediately	Not executed
2	Test the normal scenario of creating a selling post with valid data.	Created a selling post with valid data.	Required data is missing	Suspended
3				
4				
5				
...				

Verify that the marketplace can show current amounts of coin and money.

9.

Test Case ID	009	**Test Case Description**	Show current amounts of coin and money.
Created By	Kim da young	**Reviewed By**	Cho yeon jun

Step #	Prerequisites:
1	Users sell or buy the coin.
2	The database should store the coin and amount of money of the current users.
3	Show current amounts of coin and money
4	
5	
...	

Step #	Test Data Requirement
1	Coin price and money change
2	Coins and money stored in the database.
3	Visualization data:graph,chart,table.
4	Shows visualized data according to current amounts of coin and money.
5	
...	

Test Conditions

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Users sell or buy coins.	The exact current balance of the changed coins and currencies should be shown	The actual results show the accurate current amounts of coins and money.	Not executed
2	The marketplace is responsible for retrieving the current balance of coins and money from the database.	The current balance of coins and money should be retrieved.		Not executed
3	When the user checks the coins and money	The result should accurately display the current amounts of coins and money.	The actual results show the accurate current amounts of coins and money.	Not executed
4				
5				
...				

The marketplace shows the coin and money that the user currently has and verifies that the user can see it.

8. DB

1.user table - 사용자 정보

```
MariaDB [cointrade]>  CREATE TABLE users (
    ->         user_id INT AUTO_INCREMENT PRIMARY KEY,
    ->         username VARCHAR(50) NOT NULL UNIQUE,
    ->         password VARCHAR(255) NOT NULL,
    ->         email VARCHAR(100),
    ->         balance DECIMAL(15, 2) DEFAULT 100.00,
    ->         coin_balance DECIMAL(15, 6) DEFAULT 100.000000,
    ->         created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    ->     );
```

2.transactions table - 거래내역

```
MariaDB [cointrade]>  CREATE TABLE transactions (
    ->         transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    ->         user_id INT NOT NULL,
    ->         type ENUM('buy', 'sell') NOT NULL,
    ->         coin_amount DECIMAL(15, 6) NOT NULL,
    ->         price DECIMAL(15, 2) NOT NULL,
    ->         total DECIMAL(15, 2) NOT NULL,
    ->         created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    ->         FOREIGN KEY (user_id) REFERENCES users(user_id)
    ->     );
```

3.market price history - 시세 히스토리**(**코인**)**

```
MariaDB [cointrade]> CREATE TABLE market_price_history (
    ->         id INT AUTO_INCREMENT PRIMARY KEY,
    ->         coin_price DECIMAL(15, 2) NOT NULL,
    ->         recorded_at DATETIME DEFAULT CURRENT_TIMESTAMP
    ->     );
```

4.login history table - 로그인 기록

```
MariaDB [cointrade]> CREATE TABLE login_history (
    ->         login_id INT AUTO_INCREMENT PRIMARY KEY,
    ->         user_id INT NOT NULL,
    ->         login_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    ->         ip_address VARCHAR(45),
    ->         FOREIGN KEY (user_id) REFERENCES users(user_id)
    ->     );
```

5.admin logs table - 관리자 활동 로그

```
MariaDB [cointrade]>   CREATE TABLE admin_logs (
    ->         log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->         action VARCHAR(255),
    ->         performed_by VARCHAR(50),
    ->         created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    ->     );
```